

I - Exercices

I.a Chenillard

Dans ce premier exercice, vous décrirez et implémenterez votre premier circuit séquentiel sur la carte Cyclone V GX Starter Kit. Pour faire suite au TP précédent qui s'est terminé sur la synthèse d'un registre à décalage, on se propose d'implémenter un chenillard.

Créez un nouveau projet nommé TP3_chenillard

I.a.1 Description d'un compteur : Diviseur d'horloge

L'horloge que vous utiliserez pour faire défiler le chenillard a une fréquence de 50 MHz. Pour Qu'il défile à un rythme visible à l'œil nu, il est d'abord nécessaire de générer un signal d'horloge lent à l'aide d'un bloc **diviseur d'horloge**.

Vous pourrez pour ceci implémenter un module similaire aux *timers* et à leurs *prescalers* vus en cours de microcontrôleur : le module repose sur un compteur qui s'incrémente à chaque coup d'horloge, atteint une valeur d'*overflow*, et recommence un cycle de comptage ; le N ième bit du compteur fournit alors une division de l'horloge d'un facteur de 2^N

Le bloc à réaliser est décrit ainsi :

clock_divider			
	Nom	Taille	Description
Entrées	CLKin	1 bit	Horloge à l'entrée (sensibilité aux fronts montants)
	RST	1 bit	Reset du compteur, actif à l'état bas
	N	5 bits	Selection du rythme de division ($F_{out} = F_{in} / 2^{N+1}$)
Sorties	CLKout	1 bit	Horloge divisée

Travail demandé

- Ajoutez à votre projet un fichier clock_divider.vhd
- Décrivez l'*entity* clock_divider en :
 - implémentant un compteur synchrone 24 bits. (vous aurez besoin de la librairie ieee.std_logic_unsigned.all pour incrémenter un std_logic_vector)
 - multiplexant le bit N à la sortie du module
- Testez votre module en simulation

Conseil : Vous pouvez choisir pour votre simulation une valeur constante de N , suffisamment faible pour que le signal CLKout soit visible en quelques dizaines de cycles de CLKin

I.a.2 Description du chenillard

Dans cette deuxième partie, vous décrirez le comportement du chenillard.

chenillard			
	Nom	Taille	Description
Entrées	CLK	1 bit	Horloge à l'entrée (sensibilité aux fronts montants)
	RST	1 bit	Reset du chenillard
Sorties	CHEN	10 bits	Sortie du chenillard. Description des états : <pre> --- cycle ---:----CHEN--- T= 0 (reset) : 0000001111 T= 1 : 0000011110 T= 2 : 0000111100 T= 3 : 0001111100 T= 4 : 0011110000 T= 5 : 0111100000 T= 6 : 1111000000 T= 7 : 1110000001 T= 8 : 1100000011 T= 9 : 1000000111 T=10 : 0000001111 T=11 : 0000011110 ... </pre>

Travail demandé

1. Ajoutez à votre projet un fichier chenillard.vhd
2. Décrivez l'entity chenillard
3. Vérifiez son fonctionnement en simulation

I.a.3 Implémentation sur carte

D'abord, vous devez décrire comment votre module s'interface avec les entrées/sorties de la carte de développement. On appellera communément cette description le **Top Level**, puisqu'il se trouvera au sommet de la hiérarchie de votre solution.

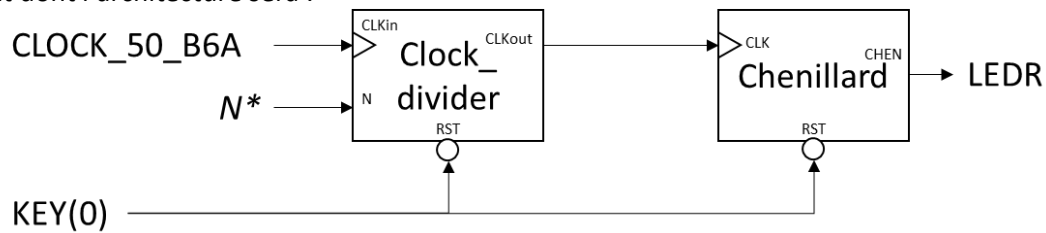
Les entrées/sorties de cette *entity* décrivent les interfaces matérielles de la carte avec lesquelles vos modules s'interfaceront. Vous pouvez lire l'intégralité de ces mots-clés dans le fichier `baseline_c5gx.v`, ajouté automatiquement à vos projets lorsque vous les créez. Ici, vous utiliserez les mots-clés réservés `CLOCK_50_B6A` (une horloge à 50 MHz) et `LEDR` (sorties vers les 10 LEDs rouges), et `KEY` (boutons poussoirs à relier aux resets).

Travail demandé

1. Ajoutez à votre projet un fichier toplevel.vhd
2. Décrivez l'entity toplevel, dont l'instruction port sera :

```
entity toplevel is
  port (
    CLOCK_50_B6A : in std_logic;
    KEY           : in std_logic_vector(3 downto 0);
    LEDR         : out std_logic_vector(9 downto 0)
  );
end entity;
```

Et dont l'architecture sera :



* : Choisir une valeur de N adéquate

3. Implémentez le chenillard sur la carte en utilisant le bouton program device.
(Voir annexe en fin de document)

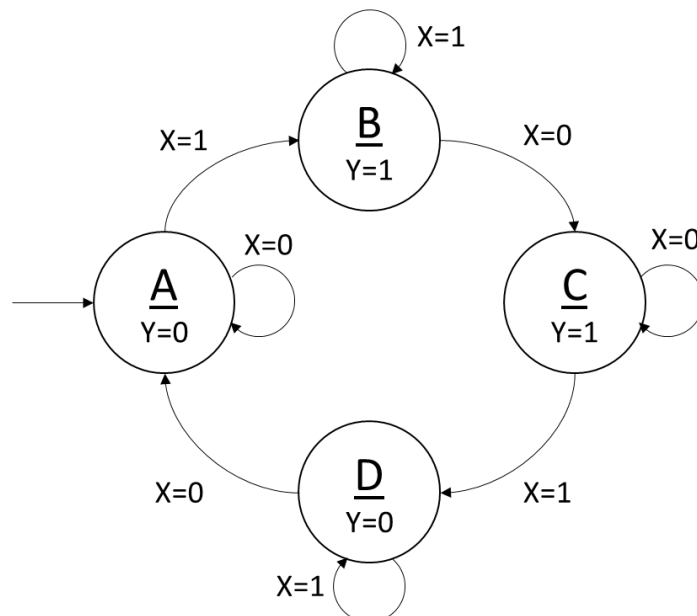
I.b Machine à états finis : bistable

On souhaite maintenant réaliser la description d'une machine à états finis. En guise de première expérience, on propose de réaliser une simple MEF à 4 états réalisant un bistable. Celui-ci sera ensuite matérialisé par l'inversion de l'état de LEDs à partir de boutons monostables.

Le module à réaliser est décrit comme suit :

bistable			
	Nom	Taille	Description
Entrées	CLK	1 bit	Horloge à l'entrée (sensibilité aux fronts montants)
	RST	1 bit	Reset (retour à l'état A)
	X	1 bit	Entrée du bistable
Sorties	Y	1 bit	Sortie du bistable

La MEF à implémenter est décrite comme suit, sous le formalisme de machine de Moore :



Vous réaliserez la description de cette MEF comme décrit dans le cours, en séparant la MEF en trois process reprenant la construction d'une machine de Moor : le registre d'états, la fonction combinatoire de calcul de l'état futur, et la fonction combinatoire de calcul des sorties.

I.b.1 Description

Travail demandé

1. Créez un nouveau projet nommé TP3_MEF, et ajoutez un nouveau fichier bistable.vhd, qui décrira l'entity bistable.
2. Copiez dans ce nouveau fichier le contenu de template_mef.vhd, disponible sur Teams.
3. Complétez la description de l'instruction port
4. Complétez dans `type state_type` les différents états du système
5. Complétez la description comportementale du registre d'états dans son process
6. Complétez la description comportementale du calcul de l'état futur dans son process
7. Complétez la description comportementale du calcul des sorties dans son process
8. Testez en simulation le bon fonctionnement du module. Portez une attention particulière aux signaux `current_state` et `future_state`.

I.b.2 Implémentation

Travail demandé

1. Créez un fichier toplevel.vhd
2. Décrivez l'entity toplevel de manière à allumer/éteindre toutes les LEDs vertes de la carte à l'appui d'un bouton en utilisant votre module bistable (1^{er} appui : ON, 2^{ème} appui : OFF, 3^{ème} appui : ON, etc.)
3. Déployez votre implémentation sur la carte

I.c Modulation du chenillard (optionnel)

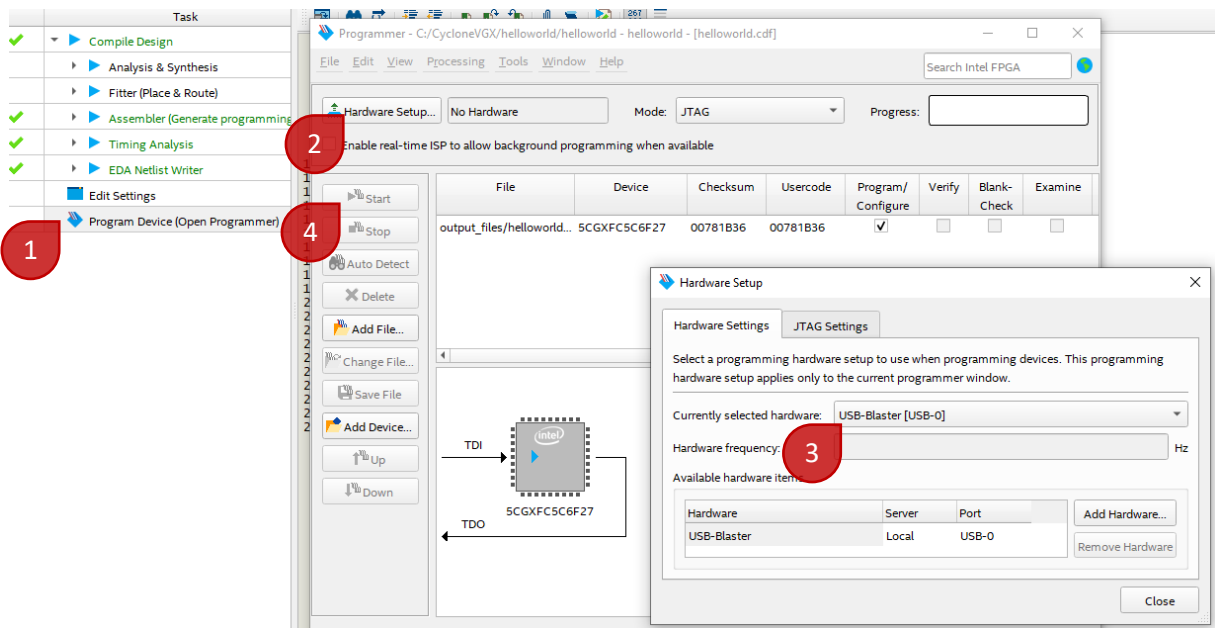
Dans un nouveau projet nommé TP3_option, on propose de décrire une version du chenillard sur laquelle vous pouvez appliquer des entrées pour en modifier le comportement.

On propose le travail suivant :

- A l'appui d'un bouton, moduler de façon bistable la vitesse de défilement du chenillard
- A l'appui d'un bouton, inverser le sens du chenillard
(cela demande l'ajout d'une entrée DIR au chenillard, indiquant sa direction de défilement)
- Défilement simultané de deux chenillards sur les LEDs rouges et vertes
(cela demande deux modules de chenillard de taille différente, vous encourageant à utiliser un *generic* – voir fin du TP2)

Annexes

Programmation de la carte :



Si vous ne trouvez pas l'USB Blaster, référez-vous au document « Annexe Quartus Prime » sur Teams